

Analyzing Space Complexity

Iterative Methods

```
Test(A[],n)
{
  int i, j;
  for ( i = 1 to n )
    A[i]=0;
}
```

$O(1)$

Iterative Methods

```
Test(A[],n)
{
  int i, j, B[n];
  for ( i = 1 to n )
    B[i]= A[i];
}
```

$O(n)$

Iterative Methods

```
Test(A[],n)
{
  int i, j, B[n][n];
  for ( i = 1 to n )
    for ( j = 1 to n )
      B[i][j]= A[i];
}
```

$O(n^2)$

Test(A[],n)

{

int i, j;

for (i = 1 to n)

 A[i]=0;

}

$O(1)$

Iterative Methods

Test(A[],n)

{

int i, j, B[n];

for (i = 1 to n)

 B[i]= A[i];

}

$O(n)$

Test(A[],n)

{

int i, j, B[n][n];

for (i = 1 to n)

 for (j = 1 to n)

 B[i][j]= A[i];

}

$O(n^2)$

Recursive Methods

(space is determined by the size of the stack)

```
demo(int n)
{
    if(n>=1)
    {
        demo(n-1);
        print (n);
    }
}
```

⇒ requires (n+1) cells

⇒ $O(n)$

Recursive Methods

(space is determined by the size of the stack)

```
demo(int n)
{
    if(n>=1)
    {
        demo(n-1);
        print (n)
        demo(n-1);
    }
}
```

⇒ requires (n+1) cells

⇒ $O(n)$

Recursive Methods

(space is determined by the size of the stack)

```
demo(int n)
{
  if(n>=1)
  {
    demo(n-1);
    print (n);
  }
}
```

```
demo(int n)
{
  if(n>=1)
  {
    demo(n-1);
    print (n);
    demo(n-1);
  }
}
```

⇒ requires (n+1) cells

⇒ O(n)